

The Search Problem

Kaiqi Zhao

Harbin Institute of Technology, Shenzhen



The Search Problem

Example. [Route planning of a robot] The only way a robot can get through a doorway is to push the door open in the direction shown. Suppose the robot starts from **o103**, and aims to get to **r123**.

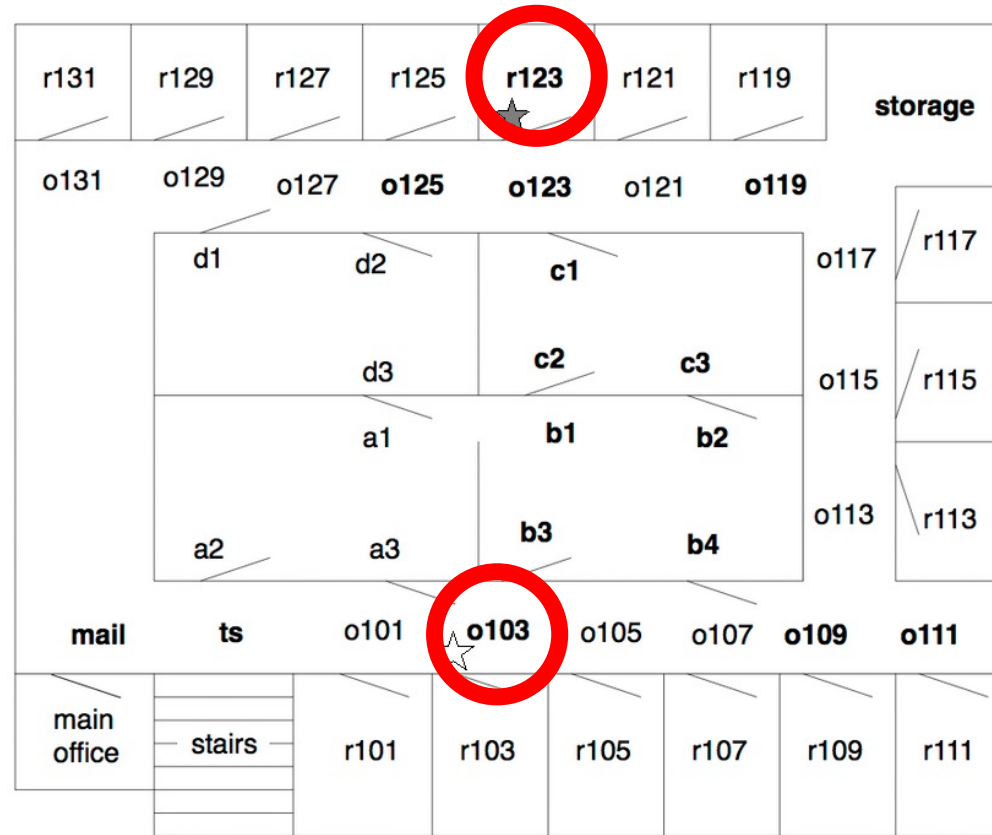
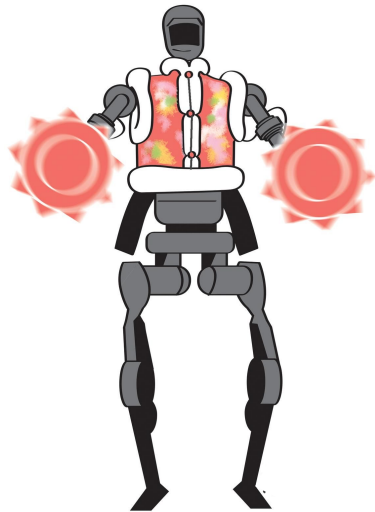
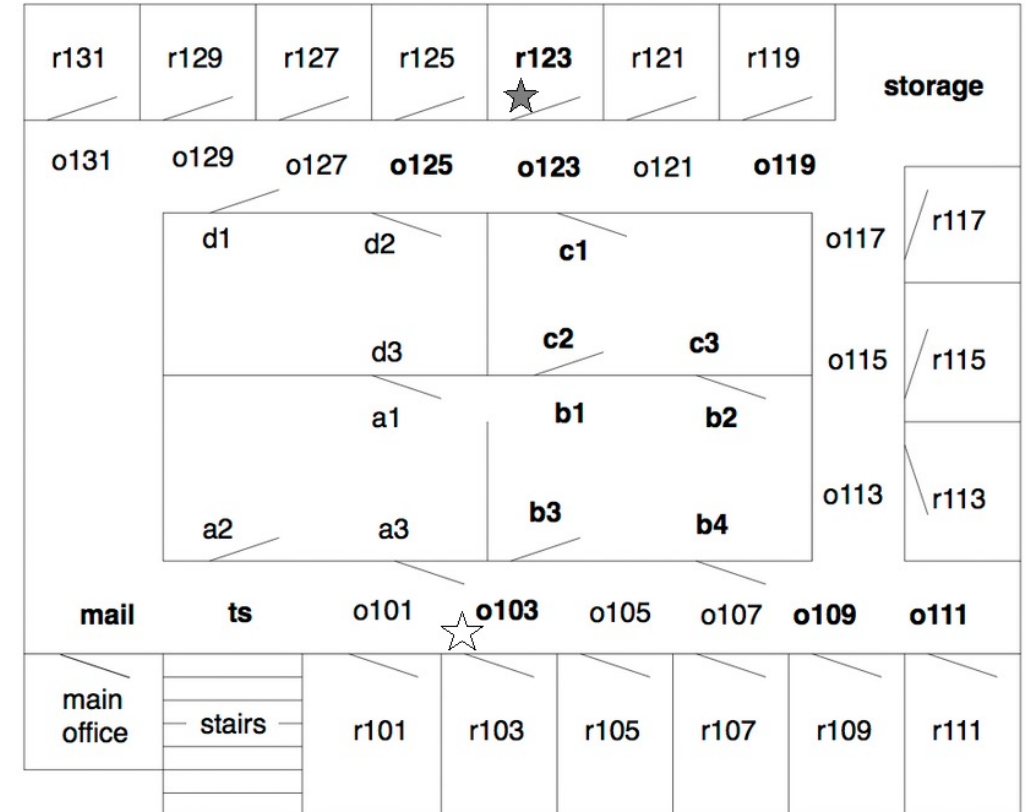


Image from <https://www.zaobao.com.sg/news/china/story20250201-5813103>

The Search Problem - Formulation

Definitions

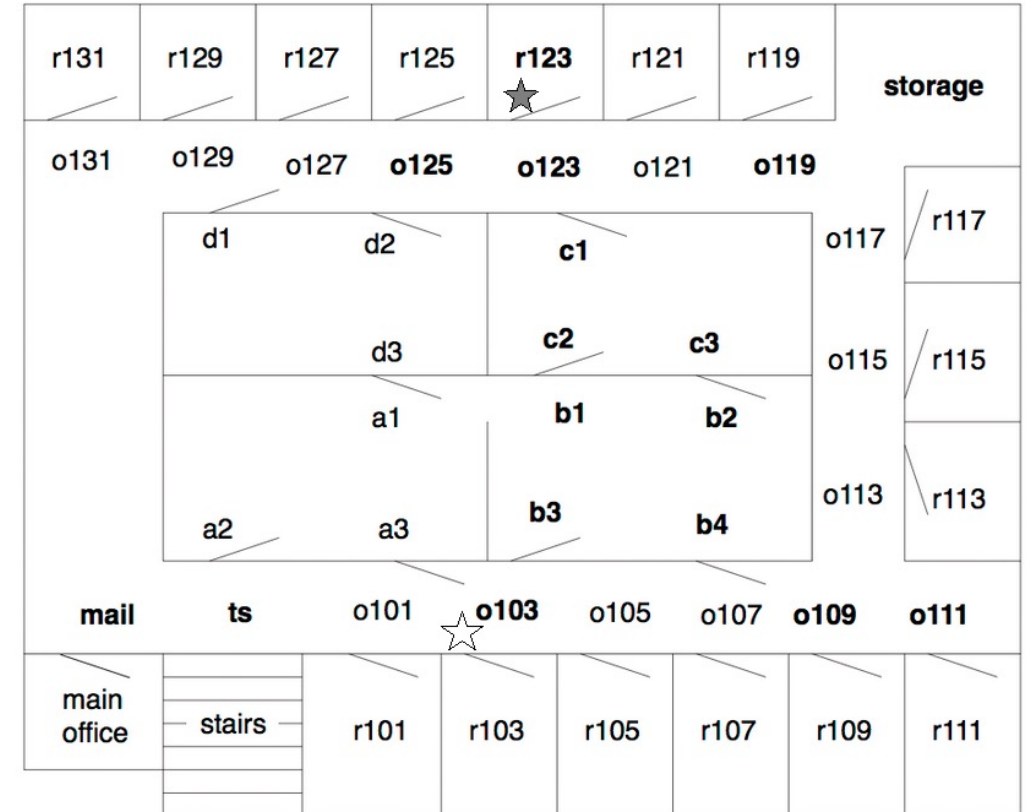
- A **state** contains all the information of the world at a specific time.
- The **state space** S contains all possible states of the world.
- Let A be the set of all actions. An **action function** $T: S \times A \rightarrow S$ that describes state transitions.
- The model is therefore represented by a **transition graph** $G: (S, E)$ whose nodes are S and edges in E are of the form (u, u') where u, u' are states, $T(u, a) = u'$ for some $a \in A$.



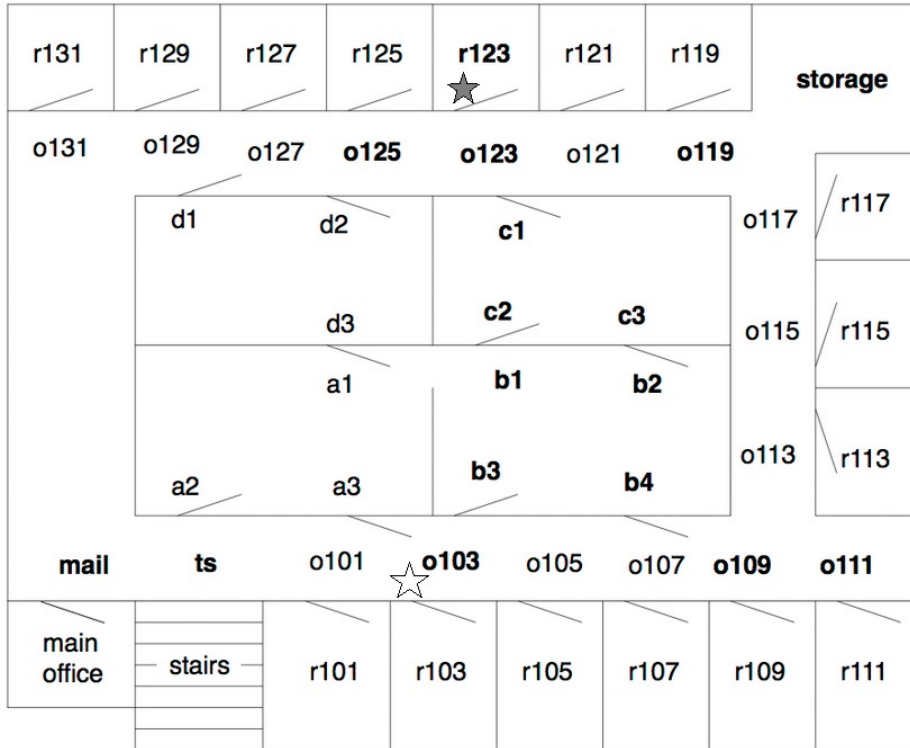
The Search Problem - Formulation

Definitions

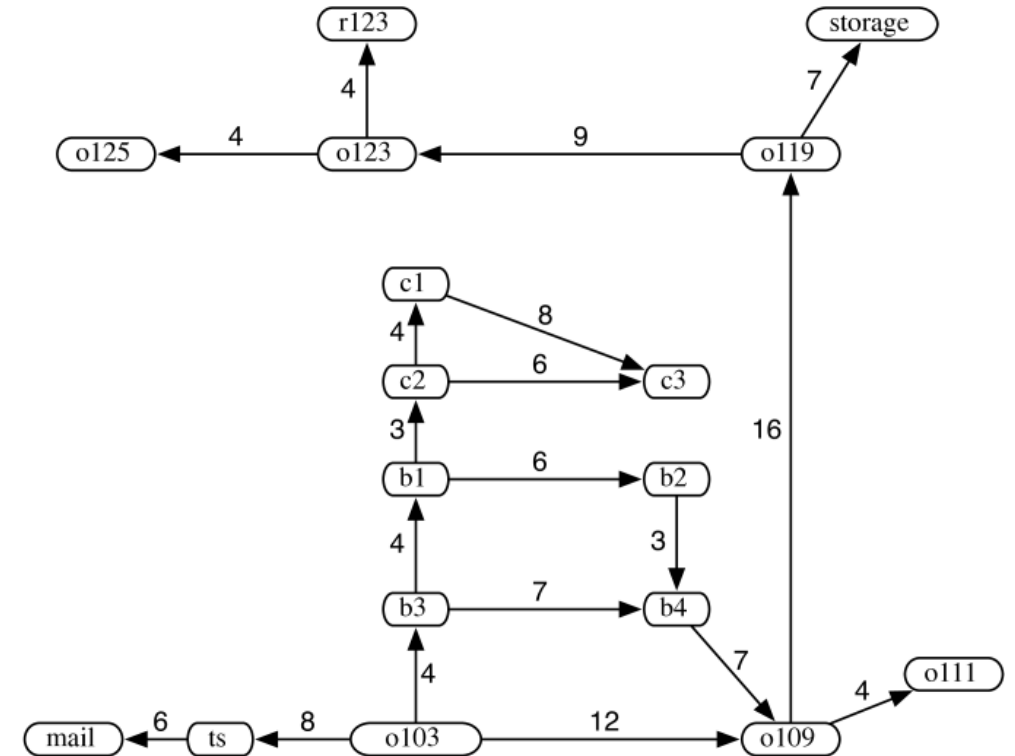
- A **path** in the transition graph is a sequence $(u_1, u_2, \dots, u_k)$ where (u_i, u_{i+1}) is an edge. A path represents a sequence of actions.
- If (u, v) is an edge in the transition graph, then v is called a **successor** of u
- A **cost function** C that assigns any path a cost value.



The Search Problem - Formulation



transition graph



Problem Definition (**State-based Search**)

INPUT State space S , action function T , start state $s \in S$, goal states $F \subseteq S$.

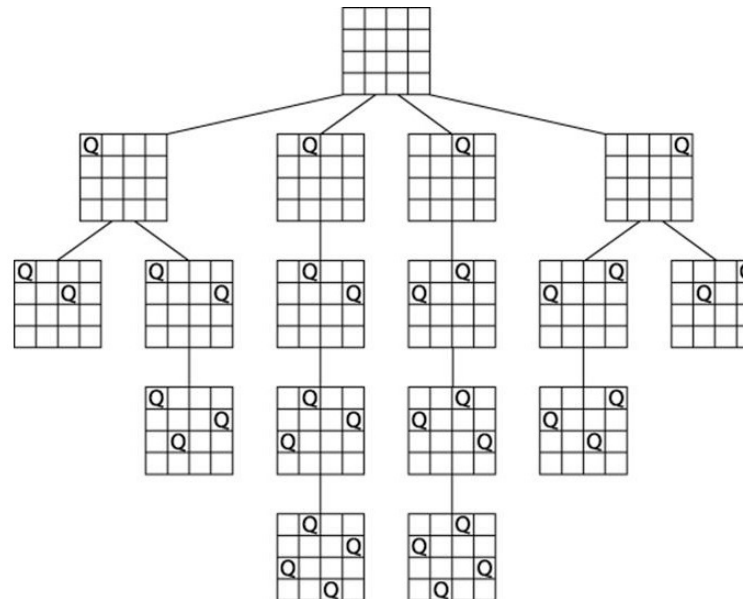
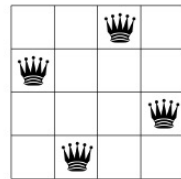
OUTPUT A path in the transition graph from s to some $s' \in F$ with the least cost.

Transition Graphs – More Examples

The transition graph is often not explicitly defined

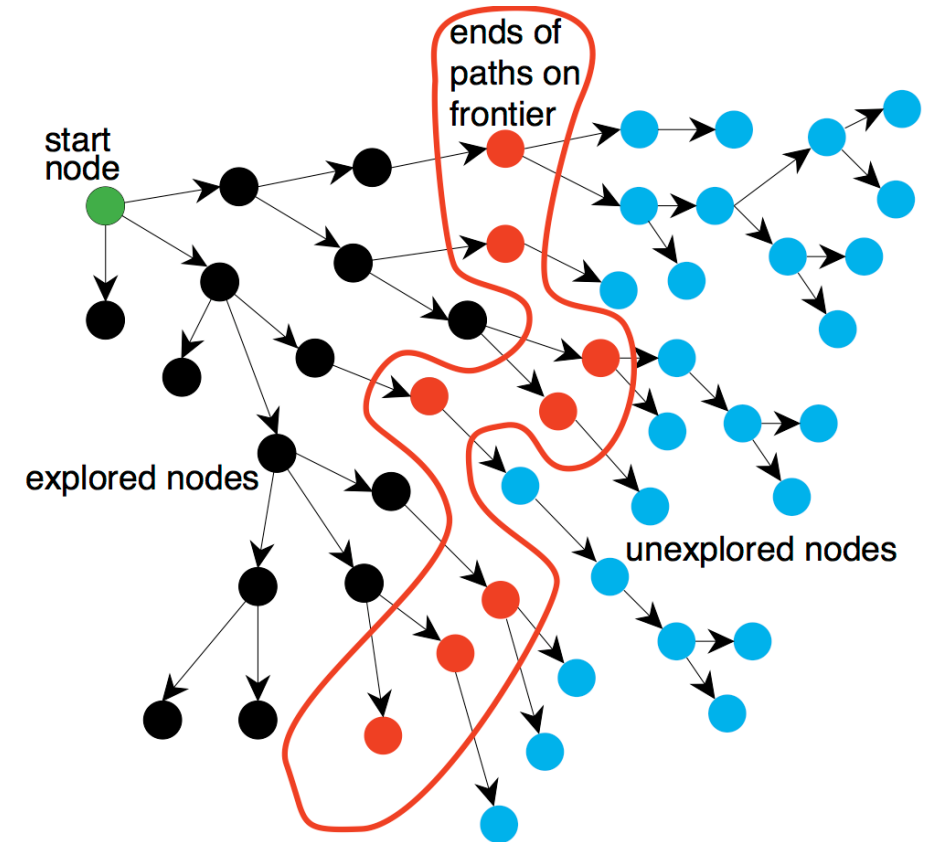
Example. [Four queens] Place four queens on a 4 x 4 chessboard, one on each column so that no two queens attack each other

- **Search space:** partial assignments where queens do not attack each other.
- **Action:** Placing one more queen in an empty column
- **Transition graph:**



A Generic Graph Search Method

- Maintaining a **frontier** that contains paths that have unexplored outgoing edges.
- Initially, the frontier only contains the start node s . The path (s) is **unexplored**
- As the search continues, the frontier expands. Once all outgoing edges of a path in frontier is checked, this path is **explored**.
- The search stops when a goal node is encountered.





The following is an algorithm **scheme** for state-based search.

Generic Search(G, s, F)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$

OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Initialise a collection **Frontier** $\leftarrow \{(s)\}$

while Frontier $\neq \emptyset$ **do**

 Select and remove a path $(s, \dots, u)$ from Frontier

if $u \in F$ **then**

 return $(s, \dots, u)$

end if

 Expand $(s, \dots, u)$ with successors v and insert the new paths into Frontier

end while

return $\perp$

Terminologies

- **Solution**: A path that ends at a goal node.
 - **Weight**: A function $w: E \rightarrow \mathbb{R}$ that denotes edge costs.
 - **The cost of a path $(u_1, \dots, u_k)$** : The sum of weights $\sum_{i=1}^{k-1} \omega(u_i, u_{i+1})$.
 - **Optimal solution**: A solution with the minimum cost.
 - **Expand a path**: Selecting and removing a path $(u_1, \dots, u_k)$ in **Frontier** and determines the successor $\{(u_1, \dots, u_k, v) \mid (u_k, v) \in E\}$ to add to **Frontier**.
 - For simplicity, we denote $\{(u_1, \dots, u_k, v) \mid (u_k, v) \in E\}$ as a successor of the path $(u_1, \dots, u_k)$ if v is a successor of u_k .
 - A **search strategy** defines the order in which paths are expanded.
- The generic search algorithm can be instantiated by specifying a search strategy.
- In the rest of the lecture we will introduce a number of search strategies.

Search Strategies



How are the successors chosen from Frontier?

Uninformed search

- The order in which the search progresses does not depend on the nature of the solution we seek

Informed search

- Knowledge about the goal and the problem domain plays a part in judging search directions

Uninformed Search

Uninformed Search Strategies

An **uninformed search strategy** does not use any a priori knowledge about the goal nodes.

Three Common Uninformed Search Strategies

1. **Depth-first search (DFS)**: Use a (LIFO) **stack** to maintain and select elements in the frontier.
2. **Breadth-first search (BFS)**: Use a (FIFO) **queue** to maintain and select elements in the frontier.
3. **Lowest-cost-first Search (LCFS)**: Use a **priority queue** to maintain and select elements in the frontier. e.g., **Dijkstra's algorithm**.

Uninformed Search 1: Depth-First Search (DFS)

DFS(G, s, F)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$

OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Initialise a **stack** **Frontier** $\leftarrow \{(s)\}$

while Frontier $\neq \emptyset$ **do**

 Pop a path $(s, \dots, u)$ from Frontier

if $u \in F$ **then**

 return $(s, \dots, u)$

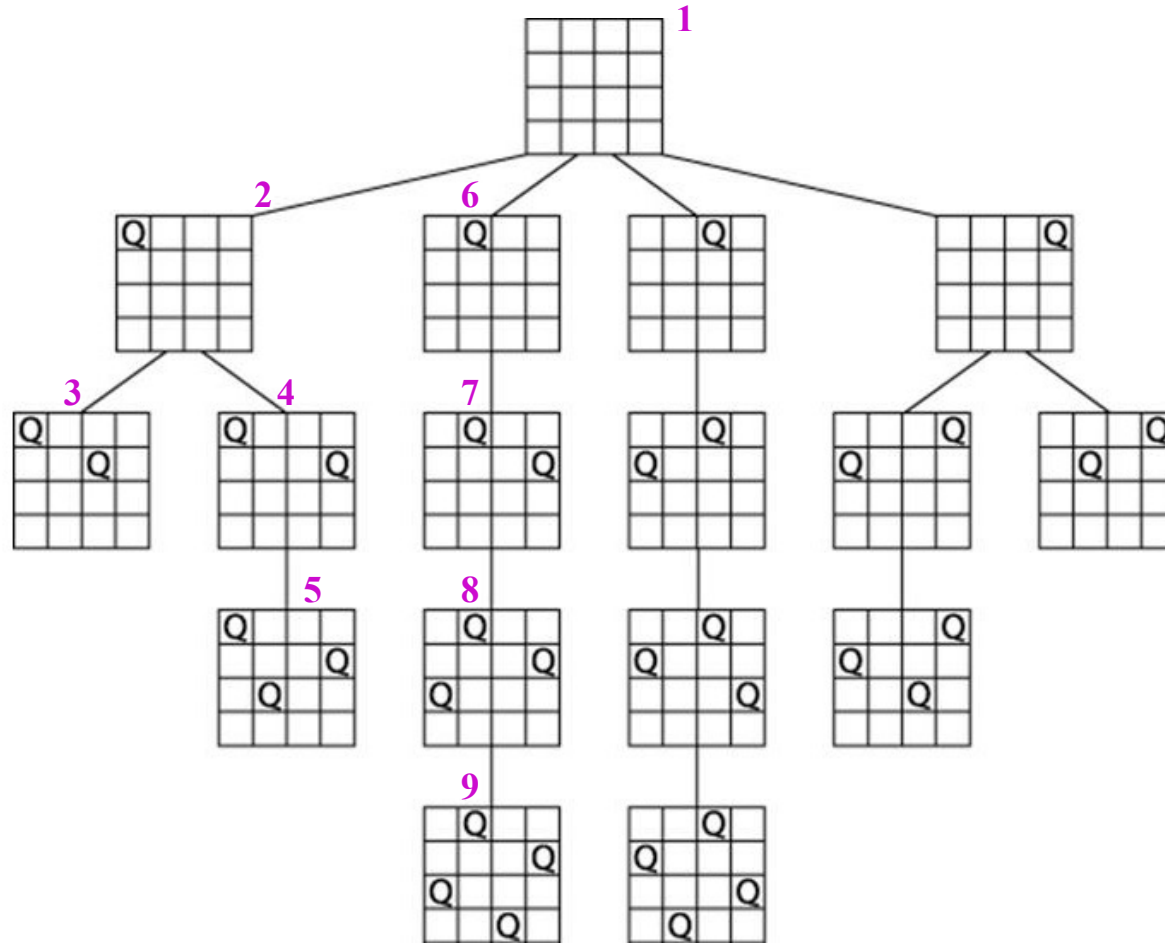
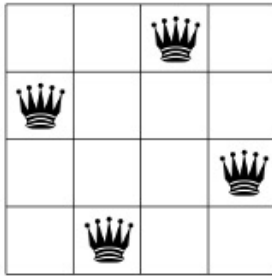
end if

 Push all successors of $(s, \dots, u)$ to Frontier

end while

return $\perp$

Example. [Four queens] Place four queens on a 4 x 4 chessboard, one on each column so that no two queens attack each other



Properties of DFS

Definitions

- The **branching factor (or degree)** of a node is the number of its successors. The branching factor of a graph is the highest branching factor of any node. Let b be the **branching factor** of the graph.
- Let m be the maximum depth of the search tree.

Time? $O(b^m)$

Space? $O(bm)$

Pros: Suitable when space is restricted and the many paths exist that lead to the goal nodes.

Cons: If the maximum depth of the graph is infinite, it is possible to get into infinite path.

Uninformed Search 2: Breadth-First Search (BFS)

BFS(G, s, F)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$

OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Initialise a **queue** **Frontier** $\leftarrow \{(s)\}$

while Frontier $\neq \emptyset$ **do**

 Dequeue a path $(s, \dots, u)$ from Frontier

if $u \in F$ **then**

 return $(s, \dots, u)$

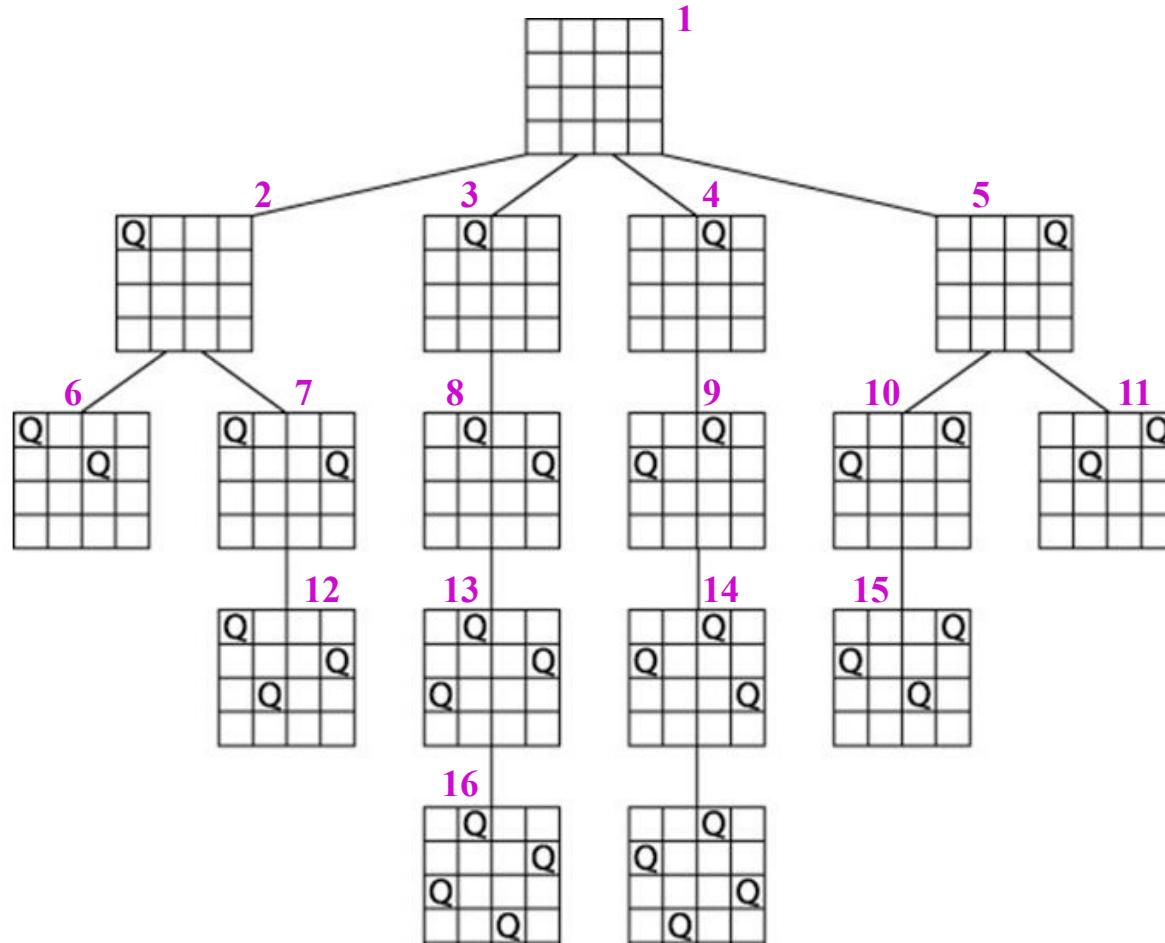
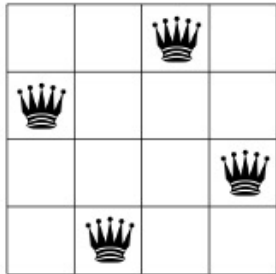
end if

 Enqueue all successors of $(s, \dots, u)$ to Frontier

end while

return $\perp$

Example. [Four queens] Place four queens on a 4 x 4 chessboard, one on each column so that no two queens attack each other



Properties of BFS

Definitions

- The **branching factor (or degree)** of a node is the number of its successors. The branching factor of a graph is the highest branching factor of any node. Let b be the **branching factor** of the graph.
- The depth of a node is the length of the shortest path from s to the node. Let k be the smallest depth of a goal node.

Time? $b + b^2 + b^3 + \dots + b^k = O(b^k)$

Space? $O(b^k)$

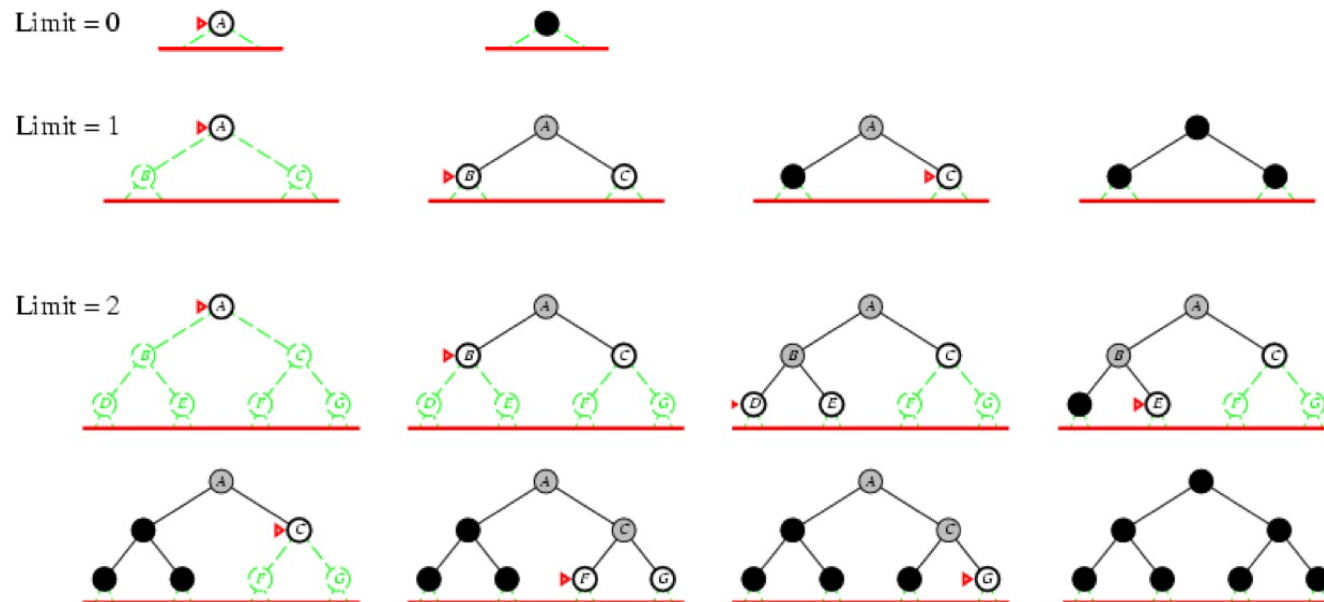
Pros: Suitable when space is not a problem and finding the path with the fewest edges is a necessity

Cons: Expensive in space requirement when the branching factor is large (many successors).

Iterative deepening

Iterative deepening is an uninformed search strategy that tries to combine the advantage of both BFS and DFS.

- It applies **depth-bounded search** to traverse through nodes up to a fixed depth using a stack.
- In the i -th iteration, it performs DFS by visiting only nodes up to depth i .



ID(G, s, F)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$

OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

limit $\leftarrow 0$

repeat

 expand $\leftarrow$ false

 res $\leftarrow$ DepthBoundedSearch(s , limit)

if res is a path **then**

 return res

end if

 limit $\leftarrow$ limit + 1

until not expand

DepthBoundedSearch($(u_0, \dots, u_k)$, limit)

INPUT: A path $(u_0, \dots, u_k)$, inter $\text{limit} \in \mathbb{Z}$

OUTPUT: a path to goal of length $k + \text{limit}$ if one exists

if limit > 0 **then**

for $(u_k, v) \in A$ **do**

 res $\leftarrow$ DepthboundedSearch($(u_1, \dots, u_k, v)$, limit - 1)

if res is a path **then**

return res

end if

end for

else if $u_k \in F$ **then**

return path $(u_1, \dots, u_k)$

else if u_k has any neighbour **then**

 expand $\leftarrow$ true

end if



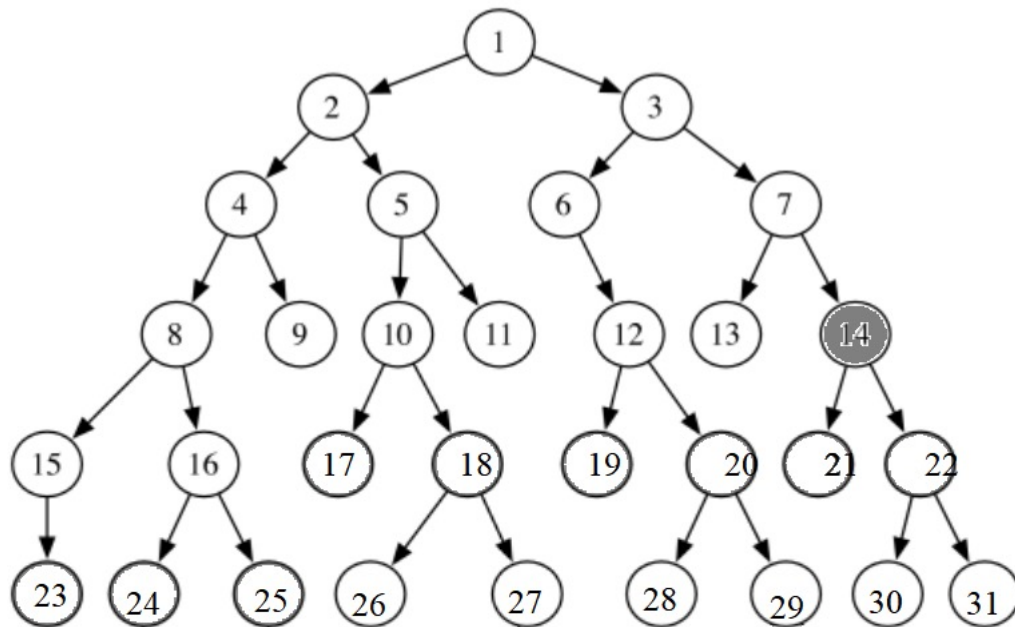
Question. What is the running time of iterative deepening?

Recall that b is the branching factor and k is the smallest depth of a goal. Suppose $b \geq 2$. The total number of paths expanded is

$$\begin{aligned} b^k + 2b^{k-1} + 3b^{k-2} + \dots + kb &= b^k(1 + 2b^{-1} + 3b^{-2} + \dots + kb^{1-k}) \\ &< b^k \left(\sum_{i=1}^{\infty} ib^{1-i} \right) \\ &= b^k \left(\frac{b}{b-1} \right)^2 \end{aligned}$$

Thus, iterative deepening has an asymptotic overhead of $\frac{b}{b-1}$ times the cost of BFS

Example. [Search algorithms] Consider the following tree. Apply BFS, DFS and ID starting from the node 1 and search for the target node (labeled by 14). Suppose that when the search has multiple directions, it will always choose the direction that has the least index.



Uninformed Search 3: Uniform-Cost Search (UCS)

- BFS, DFS and ID $\rightarrow$ no cost consideration
- UCS finds the path with **the lowest cost** to a goal node
 - At each stage, it selects the path with **the lowest cost** in the frontier.
 - The frontier is implemented as a priority queue ordered by path cost
- When each action has the same cost, UCS is equivalent to
 - BFS
- Time: $O(b^k)$ – *Need to examine every node*
- Space: $O(b^k)$ – *Need to store all the nodes*

UCS(G, s, F)

INPUT: Graph $G = (S, E)$, start node $s \in S$, goal nodes $F \subseteq S$

OUTPUT: a path from s to some node in F if exists; otherwise $\perp$

Initialise a **priority queue** **Frontier** $\leftarrow \{((s), c(s))\}$, where $c(s) = 0$

while Frontier $\neq \emptyset$ **do**

 Remove a path $(s, \dots, u)$ from Frontier with minimum cost $c(u)$

if $u \in F$ **then**

 return $(s, \dots, u)$

end if

for $v \in V$ such that $(u, v) \in E$ **do**

 If $c(u) + w(u, v) < c(v)$, then set $c(v) \leftarrow c(u) + w(u, v)$ and

 add $(s, \dots, u, v)$ into Frontier with value $c(v)$

end for

end while

return $\perp$

Search Space Pruning

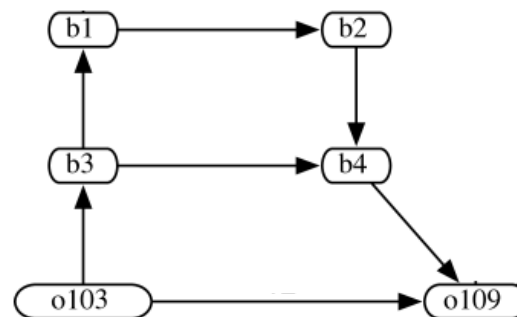
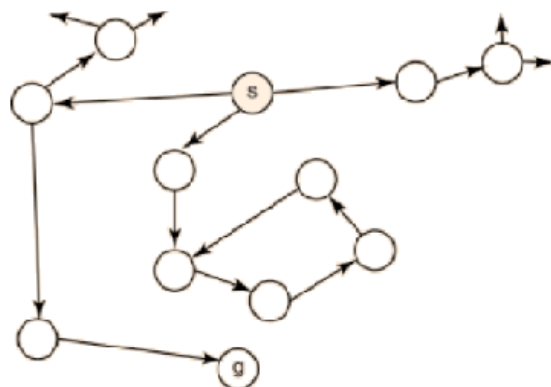


Question. Is there any general strategy that improves search efficiency?

Pruning strategies refers to methods that reduces search space in a search problem to by eliminating paths to expand.

There are two general types of pruning strategies:

- **Cycle pruning:** The generic search method may loop through cycles.
- **Multiple-path Pruning:** The generic search method maintains multiple paths between two nodes. This is not needed when only one path is asked for.

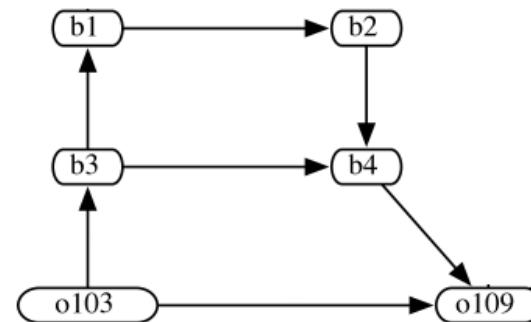
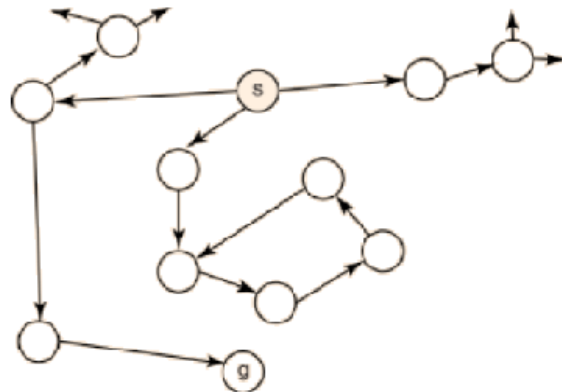


Search Space Pruning

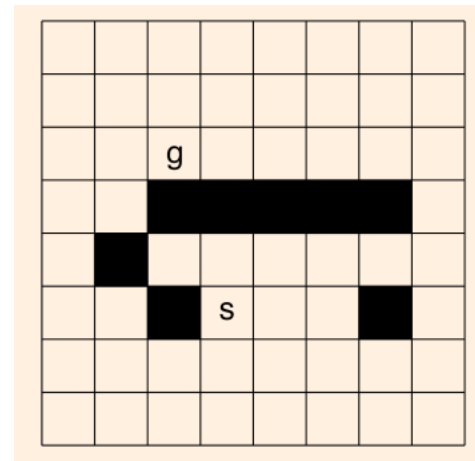
⚡ **Question.** Is there any general strategy that improves search efficiency?

Solutions:

- **Cycle pruning:** Check whether a node is in the current path before expanding the path with that node.
- **Multiple-path Pruning:** Maintain a global **explored set** containing all nodes that have been expanded. Check if a node is in the explored set before adding the path to Frontier.



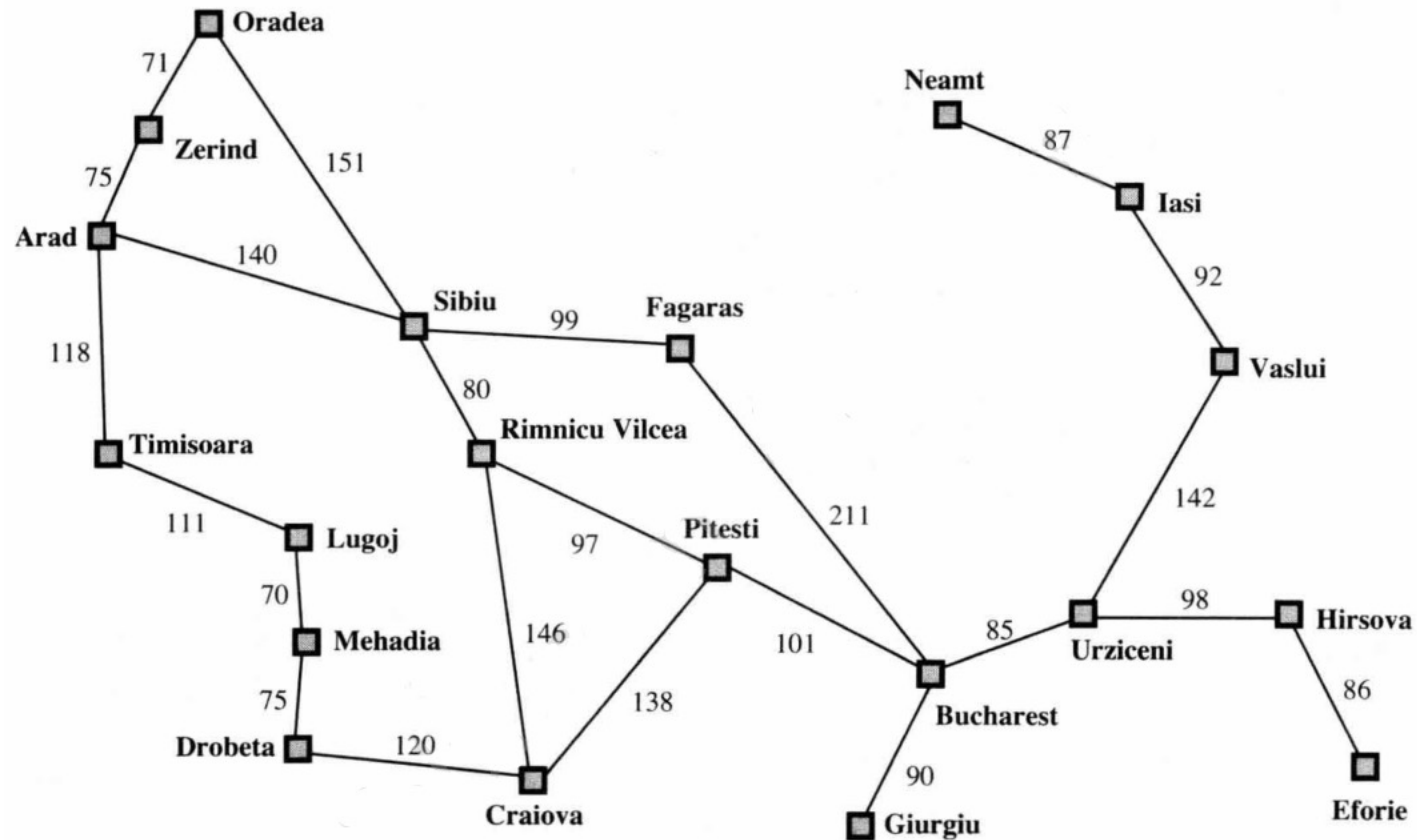
Example. [Grid problem] Consider the problem of finding a path in the grid shown below from *s* to *g*. An agent can move on the grid horizontally or vertically, one square at a time. No step may be made into a forbidden shaded area.



Number the nodes expanded (in order) for a DFS, BFS and UCS from *s* to *g*, given that the order of the operators is **up** > **left** > **right** > **down**.

Assume there is cycle pruning. What is the first path found?

Example. [Finding path with lowest cost] A tourist wants to drive to Bucharest from Arad. The map is shown in the figure below. What is the optimal (shortest) path?



Summary of The Topic

The following are the main knowledge points covered:

- What is a state-based model? What does the search problem ask for?
- What is a search strategy?
- Describe uninformed search strategies: DFS, BFS, UCS, ID.
- Discuss the computational complexity of the search algorithms.